

SQL Übungsblatt DB-interne Programmierung (functions, trigger, Transaktionen)

Alle Aufgaben beziehen sich auf die Chinook-Datenbank.

Aufgaben

1. Erstellen Sie eine Funktion, die die durchschnittliche Länge (Milliseconds) aller Tracks eines bestimmten Albums (album_id) zurückgibt.
2. Erstellen Sie eine Funktion, die die Anzahl der Tracks zurückgibt, die von dem Künstler erstellt wurden, dessen Name als Parameter übergeben wird.
3. Erstellen Sie eine Funktion, die das Einstellungsdatum (HireDate) eines Mitarbeiters (Employee) anhand der übergebenen EmployeeId zurückgibt.
4. Erstellen Sie eine Funktion, die die Gesamtzahl der Rechnungszeilen (InvoiceLine) zurückgibt, die von einem bestimmten EmployeeId betreut wurden. (Verknüpfung: Employee -> Customer -> Invoice -> InvoiceLine).

-- ab jetzt LANGUAGE PL/pgSQL --

5. Erstellen Sie eine Funktion, die prüft, ob ein Kunde (CustomerId) überhaupt Rechnungen in der Datenbank hat. Rückgabe "hat Rechnung" wenn mindestens eine Rechnung existiert, ansonsten "hat keine Rechnung".
6. Erstellen Sie eine Funktion, die die E-Mail-Adresse des Vertriebsmitarbeiters (SupportRepId) eines bestimmten Kunden (CustomerId) abrufen. Speichern Sie das Ergebnis in einer lokalen Variablen, bevor Sie es zurückgeben
7. Erstellen Sie eine Funktion, die den Rabatt auf eine Rechnung (invoice_id) berechnet. Ist "total" der Rechnung größer als 10.00, beträgt der Rabatt 10% des Gesamtbetrags; ansonsten beträgt er 5%.

-- ab jetzt TRIGGER ---

8. Erstellen sie einen Trigger und Trigger-Funktion. Bei der Aktualisierung des hire_date in der Tabelle employee soll überprüft werden, ob das neue Datum (NEW.HireDate) nach dem alten Datum (OLD.HireDate) liegt. Wenn nicht, soll eine Exception ausgelöst werden.
9. Erstellen sie einen Trigger und die dazugehörige Triggerfunktion um folgende Anforderung umzusetzen:
Beim Einfügen eines neuen Tracks in die Tabelle track soll der „media_type_id“ automatisch auf den Wert des am häufigsten verwendeten „media_type_id“ in der Datenbank gesetzt werden, falls der Wert NULL ist.

-- Transaktionen ----

10. Aufgabe REPEATABLE READ. Erstellen sie einen zwei SQL Skripte die Sie parallel ausführen. Bitte vorher in DBeaver Autocommit ausschalten. Folgender Ablauf:

1) **Beide Transaktion** mit SET TRANSACTION ISOLATION LEVEL REPEATABLE READ starten (SET SCHEMA 'chinook' nicht vergessen)

2) **Transaktion 2:**

Lesen sie den Tupel mit genre_id = 4 aus der Tabelle Genre

3) **Transaktion 1:**

Ändern sie im Tupel mit genre_id = 4 aus der Tabelle Genre den Namen auf 'RR'.
Führen sie COMMIT durch

4) **Transaktion 2:**

Lesen sie den Tupel mit genre_id = 4 aus der Tabelle Genre

Führen sie COMMIT durch

Lesen sie den Tupel mit genre_id = 4 aus der Tabelle Genre

-> erst beim Lesen **nach dem Commit in Transaktion2** sollte der geändert Wert in „name“ in Transaktion 2 sichtbar sein

11. Was ist eine Transaktion im Kontext eines Datenbanksystems, und welche vier fundamentalen ACID-Eigenschaften (Atomicity, Consistency, Isolation, Durability) muss jedes Transaktionssystem garantieren?
12. Beschreiben Sie die grundlegenden Unterschiede zwischen der Pessimistischen und der Optimistischen Concurrency-Strategie. Gehen sie auch auf das Handlings von Konflikten und die Hauptnachteile ein.

13. Erklären Sie die drei wichtigsten Concurrency-Probleme, die bei unzureichender Isolation von Transaktionen auftreten können: a) Dirty Read, b) Non Repeatable Read und c) Phantom. Welches Problem wird von der SQL-Norm grundsätzlich in allen Isolation Levels verboten?

Teil 2: Lösungen

Aufgabe 1.

```
CREATE OR REPLACE FUNCTION get_avg_track_length(id integer)
RETURNS int
LANGUAGE SQL
AS $$
    -- Berechnet den Durchschnitt der Millisekunden für die
    Tracks des Albums
    SELECT AVG(milliseconds)
    FROM track
    WHERE album_id = id;
$$;

-- Test
SELECT get_avg_track_length(1);
```

Aufgabe 2

```
CREATE OR REPLACE FUNCTION count_tracks_by_artistname(suchname
TEXT)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT COUNT(t.track_id)
    FROM track AS t
    JOIN album AS a ON t.album_id = a.album_id
    JOIN artist AS r ON a.artist_id = r.artist_id
    WHERE r.name = suchname;
$$;
```

Hinweis: wenn der Übergabeparamater suchname mit dem Typ varchar(120) definiert wird, dann funktioniert das Join-Kriterium nicht und alle Datensätze werden selektiert aus track
Lösung: Nutzung von TEXT als Typ des Übergabeparamters
Es ist unklar, warum sich postgresql so verhält ...

```
SELECT count_tracks_by_artistname('Accept');
```

Aufgabe 3

```

CREATE OR REPLACE FUNCTION get_employee_hire_date(emp_id int)
RETURNS date
LANGUAGE SQL
AS $$
    SELECT hire_date
    FROM employee
    WHERE employee_id = emp_id;
$$;

-- Test
SELECT get_employee_hire_date(3);

```

Aufgabe 4

```

CREATE OR REPLACE FUNCTION
count_invoicelines_by_employee(employee_id int)
RETURNS int
LANGUAGE SQL
AS $$
    SELECT COUNT(il.invoice_line_id)
    FROM employee AS e
    JOIN customer AS c ON e.employee_id = c.support_rep_id
    JOIN invoice AS i ON c.customer_id = i.customer_id
    JOIN invoice_line AS il ON i.invoice_id = il.invoice_id
    WHERE e.employee_id = employee_id;
$$;

SELECT count_invoicelines_by_employee(3);

```

Aufgabe 5

```

CREATE OR REPLACE FUNCTION KundeRechnung(cust_id integer)
RETURNS VARCHAR(20)
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (SELECT * FROM invoice WHERE customer_id = cust_id)
    THEN
        RETURN 'hat Rechnung';
    ELSE
        RETURN 'hat keine Rechnung';
    END IF;

```

```
END;  
$$;
```

Test mit:

```
SELECT customer_id, first_name, last_name,  
kunderechnung(customer_id) FROM customer;
```

Aufgabe 6

```
CREATE OR REPLACE FUNCTION supportemail(cust_id integer)  
RETURNS text  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    support_email text;  
BEGIN  
    SELECT e.email INTO support_email  
    FROM customer AS c  
    JOIN employee AS e ON c.support_rep_id = e.employee_id  
    WHERE c.customer_id = cust_id;  
  
    RETURN support_email;  
END;  
$$;
```

-- Test

```
SELECT customer_id, first_name, last_name,  
supportemail(customer_id) AS emailsupport  
FROM customer;
```

Aufgabe 7

```
CREATE OR REPLACE FUNCTION discount(invoice_id_param integer)  
RETURNS numeric  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    invoice_total numeric;  
    discount_amount numeric := 0.00;  
BEGIN
```

```

SELECT total INTO invoice_total
FROM invoice
WHERE invoice_id = invoice_id_param;

IF invoice_total > 10.00 THEN
    discount_amount := invoice_total * 0.10;
ELSE
    discount_amount := invoice_total * 0.05;
END IF;

RETURN discount_amount;
END;
$$;

-- TEST
SELECT invoice_id, total, discount(invoice_id) AS Rabatt
FROM invoice;

```

Aufgabe 8

```

CREATE OR REPLACE FUNCTION check_hire_date()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Prüfen, ob neues Einstellungsdatum vor dem alten liegt
    IF NEW.hire_date < OLD.hire_date THEN
        RAISE EXCEPTION 'Das Einstellungsdatum kann nicht...';
    END IF;
    RETURN NEW;
END;
$$;

-- Trigger (spaltenspezifisch):
CREATE TRIGGER check_date_forward
BEFORE UPDATE OF hire_date ON employee
FOR EACH ROW
EXECUTE FUNCTION check_hire_date();

-- check
UPDATE employee
SET hire_date = '01.01.1900'
WHERE employee_id = 1;

```

Aufgabe 9

```
CREATE OR REPLACE FUNCTION default_media_type()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    -- Lokale Variable für die häufigste Media Type ID
    most_used_media_id int;
BEGIN
    -- Nur ausführen, wenn der Wert nicht gesetzt wurde
    IF NEW.media_type_id IS NULL THEN

        -- Ermittelt die am häufigsten verwendete Media Type ID
        SELECT media_type_id INTO most_used_media_id
        FROM track
        GROUP BY media_type_id
        ORDER BY COUNT(*) DESC
        LIMIT 1;

        -- Zuweisen des Standardwerts
        NEW.media_type_id := most_used_media_id;

    END IF;
    RETURN NEW;
END;
$$;

-- Trigger:
CREATE TRIGGER default_media_type_trigger
BEFORE INSERT ON track
FOR EACH ROW
EXECUTE FUNCTION default_media_type();

-- CHECK
-- Datensatz einfügen ohne media_type_id
INSERT INTO track (track_id, name, album_id, genre_id, composer,
millisecons, bytes, unit_price) VALUES
    (8888, N'For Those About To Rock (We Salute You)', 1, 1,
N'Angus Young, Malcolm Young, Brian Johnson', 343719, 11170334,
0.99);

SELECT * FROM track
WHERE track_id = 8888;
```

Aufgabe 10

```
-- SESSION 1
```



```

SET SCHEMA 'chinook';
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- SESSION 2
SET SCHEMA 'chinook';
START TRANSACTION;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT * FROM genre WHERE genre_id = 5;

-- SESSION 1
UPDATE genre SET name = 'RR' WHERE genre_id = 5;
COMMIT;

-- SESSION 2
SELECT * FROM genre WHERE genre_id = 5;
-- Änderung trotz COMMIT in Session 1 nicht in Session 2 sichtbar
-- SESSION 2 JETZT COMMIT in Session 2 und erneut lesen --
COMMIT;
SELECT * FROM genre WHERE genre_id = 5;
-- Änderung ist erst jetzt in Transaktion 2 sichtbar

```

Aufgabe 11

Eine Transaktion ist eine logische Einheit von Datenbankankweisungen. Sie überführt die Datenbank von einem konsistenten Zustand in einen neuen, ebenfalls konsistenten Zustand. Die ordnungsgemäße Beendigung dieser Transaktionen wird durch das Transaktionsmanagement des DBMS kontrolliert, welches die ACID-Eigenschaften (Atomicity, Consistency, Isolation, Durability) garantieren muss.

Atomicity (Atomarität): Diese Eigenschaft stellt sicher, dass eine Transaktion entweder vollständig ausgeführt und umgesetzt wird, oder gar nicht. Der altgriechische Ursprung des Wortes soll ausdrücken, dass die Anweisungen der Transaktion nicht weiter

aufteilbar sind. Bei einem Fehler oder einem Abbruch stellt das System sicher, dass alle seit Beginn der Transaktion vorgenommenen Einzeländerungen rückgängig gemacht werden (ROLLBACK), um den ursprünglichen konsistenten Zustand wiederherzustellen.

Consistency (Konsistenz): Die Konsistenz garantiert, dass die Datenbank nach erfolgreichem Abschluss der Transaktion alle vordefinierten Regeln, Integritätsbedingungen und Geschäftsvorgaben einhält. Wenn die Datenbank vor der Transaktion konsistent war, sorgt ein erfolgreiches COMMIT für einen konsistenten Zustand danach. Sollte die Verarbeitung unterbrochen werden, stellt der ROLLBACK die Ausgangskonsistenz wieder her..

Isolation (Isolation): Die Isolation gewährleistet, dass gleichzeitig ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Die Ergebnisse der Anweisungen innerhalb der Transaktion bleiben bis zum COMMIT für andere Transaktionen unsichtbar, sodass jede Transaktion so abläuft, als wäre sie die einzige im System. Der genaue Grad der Isolation wird durch die Isolation Level festgelegt.

Durability (Dauerhaftigkeit): Nach dem erfolgreichen Abschluss einer Transaktion durch COMMIT müssen die vorgenommenen Änderungen dauerhaft und permanent in der Datenbank gespeichert sein. Die Daten müssen auch bei einem anschließenden Systemabsturz oder Ausfall erhalten bleiben (über Log-Einträge gesichert).

Aufgabe 12

Die Pessimistische Concurrency-Strategie beruht auf Sperrmechanismen (Locks). Wenn eine Transaktion auf Daten zugreift (auch nur lesend, je nach Isolationsgrad), werden diese Daten sofort für andere Transaktionen gesperrt. Diese Sperren werden erst bei Transaktionsende (COMMIT/ROLLBACK) freigegeben. Dies erzwingt die Serialisierung, hat aber den Nachteil, dass es zu Wartezeiten kommt und die Gefahr von Verklemmungen (Deadlocks) besteht.

Die Optimistische Concurrency-Strategie erlaubt Transaktionen, Änderungen vorzunehmen, da sie davon ausgeht, dass Konflikte selten sind. Die Konfliktprüfung erfolgt erst bei der Validierung kurz vor dem COMMIT. Wird zu diesem Zeitpunkt festgestellt, dass andere Transaktionen dieselben Daten bearbeitet und committed haben, wird die aktuelle Transaktion zurückgenommen (ROLLBACK) und muss neu starten. Obwohl dies zu guter Performance bei niedriger Konfliktrate führt, kann die Notwendigkeit des Neustarts bei hoher Parallelität die Vorteile schnell aufheben.

Aufgabe 13

Dirty Read (Lesen "schmutziger" Daten): Dies ist das Problem der Abhängigkeit von noch nicht abgeschlossenen Transaktionen. Transaktion A liest Daten, die von

Transaktion B zwar geändert, aber noch nicht durch COMMIT dauerhaft festgeschrieben wurden. Sollte Transaktion B diese Änderung später durch ROLLBACK zurücknehmen, hat A auf ungültigen Daten gearbeitet, was zu Inkonsistenzen führen kann.

Non Repeatable Read (Nicht wiederholbares Lesen): Bei diesem Problem wird nicht garantiert, dass ein einmal gelesener Eintrag innerhalb derselben Transaktion unverändert bleibt. Wenn Transaktion A einen Datensatz liest und eine andere Transaktion B diesen Datensatz ändert und commitet, erhält A bei einem wiederholten Lesen desselben Datensatzes innerhalb der eigenen Transaktion ein unterschiedliches Ergebnis.

Phantom (Phantomproblem): Diese Anomalie tritt auf, wenn eine Transaktion A eine Abfrage wiederholt, die auf einer Menge von Zeilen basiert (z.B. SELECT COUNT(*) oder eine Bereichsabfrage), und beim wiederholten Lesen neue Zeilen auftauchen. Diese neuen Zeilen, Phantome genannt, wurden von einer zwischenzeitlich abgeschlossenen Transaktion B eingefügt und committed.

Das Problem des Dirty Write (oder auch das Problem der verlorengegangenen Änderung – Lost Update) wird von der SQL-Norm grundsätzlich in allen Isolation Levels verboten. Ein Dirty Write liegt vor, wenn die Änderung einer Transaktion durch die Änderung einer zweiten, parallel laufenden Transaktion überschrieben wird, bevor die erste Transaktion committed wurde, wodurch die Änderung der ersten Transaktion verloren geht.